Résumé complet : les listes chaînées en C

Fiche de révision pour le contrôle de lundi, construite à partir du cours *Structures de données* (chapitre « Les listes chaînées en C »).

1. Pourquoi les listes chaînées ?

Le problème des tableaux

Un tableau est un seul bloc de mémoire avec des cases collées les unes aux autres, et sa taille est fixe.

- Accès à un élément : **O(1)**, car on calcule directement l'adresse.
- Recherche : O(n).
- Ajout ou suppression au milieu : O(n), car il faut décaler les autres éléments.

La solution : la liste chaînée

Chaque élément (appelé **maillon**, **cellule** ou **nœud**) est alloué séparément, et les éléments sont éparpillés en mémoire. Ce qui les relie, ce sont des **pointeurs** : chaque maillon sait où se trouve le suivant.

Opération	Tableau	Liste chaînée
Taille	Fixe, connue au départ	Variable, change pendant l'exécution
Mémoire	Un seul bloc contigu	Maillons éparpillés
Accès à un élément	O(1)	O(n) (il faut parcourir)
Recherche	O(n)	O(n)
Ajout	O(n)	O(1) au début
Suppression	O(n)	O(1) si on connaît déjà le maillon (selon le cas)

À retenir : la liste chaînée est excellente pour insérer et supprimer, et mauvaise pour accéder directement au i-ème élément.

Les types de listes

1. **Liste simplement chaînée** : chaque maillon a un pointeur vers le suivant.

2. **Liste doublement chaînée** : chaque maillon a deux pointeurs, vers le précédent et vers le suivant.
3. **Liste circulaire** : le dernier élément pointe vers le premier (et, si elle est doublement chaînée, le premier pointe aussi vers le dernier).

Définition récursive d'une liste

Une liste est soit **vide**, soit composée d'une **tête** (la valeur du premier élément) et d'une **queue** (le reste de la liste, qui est elle-même une liste).

2. Les prérequis à bien maîtriser (pointeurs)

Sans les pointeurs, les listes chaînées sont incompréhensibles. Voici l'essentiel.

- Un **pointeur** est une variable qui contient une **adresse mémoire**.
- `int *p;` déclare un pointeur sur un entier.
- `p = &x;` : `&x` est l'adresse de `x`, donc `p` pointe sur `x`.
- `*p` donne la **valeur** à l'adresse pointée (déréférencement). `*p = 20;` modifie `x`.
- Un pointeur qui ne pointe sur rien vaut **NULL**.
- **Flèche `->`** : si `p` est un pointeur sur une structure, `p->champ` est un raccourci pour `(*p).champ`.
- **Allocation dynamique** : `malloc(sizeof(Node))` réserve de la mémoire et renvoie son adresse. Il faut tester si le résultat vaut `NULL` (échec) et libérer avec `free(p)` quand on n'en a plus besoin.

Règle d'or : ne jamais déréférencer un pointeur NULL (`p->next` quand `p == NULL` fait planter le programme).

3. Les listes simplement chaînées

3.1 Le principe

- Chaque maillon contient des **données** et un pointeur **`next`** vers le maillon suivant.
- Un pointeur **`head`** (la tête) pointe sur le premier maillon.
- Le dernier maillon a `next = NULL` : c'est ce qui marque la **fin de liste**.
- Liste vide : `head = NULL`.

Schéma de la liste 3, 81, 19 :

```
head --> [3 | next] --> [81 | next] --> [19 | NULL]
```

3.2 Définir un nœud

La structure se référence elle-même (par un pointeur), d'où le `struct node *next` à l'intérieur de la définition :

```
typedef struct node {
    int val;            // les données
    struct node *next;  // pointeur vers le suivant
} Node;
```

Les données peuvent être de n'importe quel type : plusieurs champs (`char nom[20]; char prenom[20]; int age;`) ou même un enregistrement complet (`Player player;`). Le principe reste identique : **données + `next`**.

3.3 Créer une liste vide

```
Node *head = NULL;
```

3.4 Créer (allouer) un nœud

```
Node* creerNoeud(int valeur) {
    Node* nouveau = (Node*) malloc(sizeof(Node));
    if (nouveau == NULL) {
        printf("Erreur d'allocation mémoire\n");
        exit(1);
    }
    nouveau->val = valeur;
    nouveau->next = NULL;
    return nouveau;
}
```

À chaque fois on fait : allouer, tester l'échec, remplir `val`, mettre `next` à `NULL`, renvoyer l'adresse.

3.5 Insérer au début

Étapes :

1. Allouer un nouveau nœud.
2. Son `next` doit pointer sur l'ancienne tête.
3. La nouvelle tête devient ce nœud.

```
Node* insererDebut(Node* head, int valeur) {
    Node* nouveau = creerNoeud(valeur);
    nouveau->next = head;
    return nouveau;   // nouvelle tête
}
```

Utilisation : `head = insererDebut(head, 40);`

Pourquoi on retourne la tête ? La fonction travaille sur une copie du pointeur `head` (passage par valeur). Si la tête change, on doit la renvoyer et la récupérer avec `head = ...`. Oublier ce `head =` est une erreur classique.

Complexité : O(1), aucun parcours.

3.6 Insérer à la fin

Étapes :

1. Allouer un nouveau nœud.
2. Si la liste est vide, le nouveau nœud devient la liste.
3. Sinon, parcourir jusqu'au **dernier** nœud (celui dont `next == NULL`).
4. Faire pointer le `next` du dernier sur le nouveau nœud.

```
Node* insererFin(Node* head, int valeur) {
    Node* nouveau = creerNoeud(valeur);
    if (head == NULL) return nouveau;     // liste vide
    Node* temp = head;
    while (temp->next != NULL)            // on s'arrête SUR le dernier
        temp = temp->next;
    temp->next = nouveau;
    return head;
}
```

Complexité : O(n) (parcours complet), sauf si on garde un pointeur `tail` vers le dernier maillon.

Attention à la condition : pour insérer à la fin on utilise `temp->next != NULL` (on veut s'arrêter *sur* le dernier maillon), alors que pour simplement parcourir on utilise `temp != NULL`.

3.7 Insérer après une valeur donnée

Étapes : chercher le nœud qui contient la clé, puis intercaler le nouveau nœud juste après.

```
Node* insererApres(Node* head, int valeur, int cle) {
    Node* temp = head;
    while (temp != NULL && temp->val != cle)
        temp = temp->next;
    if (temp == NULL) {
        printf("Valeur %d non trouvée.\n", cle);
        return head;
    }
    Node* nouveau = creerNoeud(valeur);
    nouveau->next = temp->next;   // 1) le nouveau pointe sur l'ancien
suivant
    temp->next = nouveau;         // 2) le nœud trouvé pointe sur le nouveau
    return head;
}
```

L'ordre des deux dernières lignes est crucial. Si on fait `temp->next = nouveau` en premier, on perd l'accès au reste de la liste. Retiens : *d'abord on accroche le nouveau à la suite, ensuite on le branche à la liste.*

3.8 Supprimer un élément

Étapes :

1. Si la valeur à supprimer est dans le **premier** nœud, `head` passe au nœud suivant.
2. Sinon, chercher le nœud **précédent** de celui à supprimer.
3. Faire pointer son `next` sur le suivant du nœud à supprimer (on « contourne » le nœud).
4. **Libérer** la mémoire du nœud supprimé avec `free`.

```
Node* supprimerValeur(Node* head, int cle) {
    if (head == NULL) return NULL;
    if (head->val == cle) {             // cas : premier élément
        Node* temp = head;
        head = head->next;
        free(temp);
        return head;
    }
    Node* temp = head;
    while (temp->next != NULL && temp->next->val != cle)
        temp = temp->next;
    if (temp->next == NULL) {
        printf("Valeur %d non trouvée.\n", cle);
        return head;
    }
    Node* aSupprimer = temp->next;
```

```
    temp->next = aSupprimer->next;       // on contourne
    free(aSupprimer);
    return head;
}
```

Idée clé : pour supprimer un maillon dans une liste simple, il faut s'arrêter sur celui **d'avant**, car on ne peut pas revenir en arrière.

3.9 Parcourir et afficher

```
void afficherListe(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->val);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

On utilise un pointeur temporaire (`temp`) pour ne **jamais perdre** **`head`** . Toutes les opérations de parcours suivent ce même modèle.

3.10 Longueur d'une liste

```
int longueurListe(Node* head) {
    int count = 0;
    Node* temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}
```

3.11 Exemple complet (à savoir tracer)

```
int main() {
    Node* liste = NULL;
    liste = insererDebut(liste, 3);   // 3
    liste = insererDebut(liste, 2);   // 2 -> 3
    liste = insererDebut(liste, 1);   // 1 -> 2 -> 3
    liste = insererFin(liste, 4);     // 1 -> 2 -> 3 -> 4
    afficherListe(liste);             // 1 -> 2 -> 3 -> 4 -> NULL
    liste = supprimerValeur(liste, 2);// 1 -> 3 -> 4
    afficherListe(liste);             // 1 -> 3 -> 4 -> NULL
    printf("Longueur : %d\n", longueurListe(liste));  // 3
    return 0;
}
```

Le piège : `insererDebut` met chaque nouvel élément **devant**, donc l'ordre d'insertion 3, 2, 1 donne la liste `1 -> 2 -> 3`.

3.12 Les exercices du cours (avec leur logique)

Compter les occurrences d'une valeur : on parcourt la liste et on incrémente un compteur à chaque fois que `temp->val == valeur`.

```
int compter(Node* head, int valeur) {
    int compteur = 0;
    Node* temp = head;
    while (temp != NULL) {
        if (temp->val == valeur) compteur++;
        temp = temp->next;
    }
    return compteur;
}
```

Trouver le maximum : on initialise `max` avec la première valeur, puis on parcourt le reste (la liste ne doit pas être vide).

```
int maximum(Node* head) {
    int max = head->val;
    Node* temp = head->next;
    while (temp != NULL) {
        if (temp->val > max) max = temp->val;
        temp = temp->next;
    }
    return max;
}
```

Concaténer deux listes : si `L1` est vide on renvoie `L2`. Sinon on va au dernier nœud de `L1` et on lui accroche `L2`.

```
Node* concatener(Node* L1, Node* L2) {
    if (L1 == NULL) return L2;
    Node* temp = L1;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = L2;
    return L1;
}
```

4. Les listes doublement chaînées

4.1 Principe

Chaque maillon possède **deux pointeurs** : `prev` (précédent) et `next` (suivant). On parle aussi de liste *symétrique*.

- Le `prev` du **premier** maillon vaut `NULL`.
- Le `next` du **dernier** maillon vaut `NULL`.

```
NULL <-- [prev|3|next] <--> [prev|81|next] <--> [prev|19|next] --> NULL
          ^ head
```

Deux variantes existent : avec seulement `head`, ou avec `head` **et** `tail` (pointeur sur le dernier). Avec `tail`, l'ajout en fin devient O(1) et on peut parcourir à l'envers.

4.2 Le nœud

```
typedef struct node {
    struct node *prev;
    int val;               // données
    struct node *next;
} Node;
```

4.3 Les primitives (le cours les présente par schémas)

Le cours illustre trois opérations par des dessins : ajouter à la fin, insérer un élément, supprimer un élément. Voici la logique et un code type pour chacune.

Ajouter à la fin : on se place sur le dernier nœud, puis on relie dans les **deux sens**.

```
Node* insererFin(Node* head, int valeur) {
    Node* nouveau = creerNoeud(valeur);   // prev = NULL, next = NULL
    if (head == NULL) return nouveau;
    Node* temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = nouveau;   // le dernier pointe sur le nouveau
    nouveau->prev = temp;   // le nouveau pointe en arrière sur l'ancien
dernier
    return head;
}
```

Insérer après un nœud `cur` : il y a **quatre liens** à mettre à jour (deux pour le nouveau, deux pour ses voisins).

```
nouveau->next = cur->next;
nouveau->prev = cur;
if (cur->next != NULL) cur->next->prev = nouveau;
cur->next = nouveau;
```

Supprimer un nœud `cur` : on relie ses deux voisins entre eux, puis on libère.

```
if (cur->prev != NULL) cur->prev->next = cur->next;
else head = cur->next;                    // on supprime la tête
if (cur->next != NULL) cur->next->prev = cur->prev;
free(cur);
```

4.4 Avantages et inconvénients

- **Avantage :** on peut parcourir dans les deux sens, et supprimer un maillon dont on a l'adresse **sans chercher le précédent** (il est dans `cur->prev`).
- **Inconvénient :** chaque maillon occupe plus de mémoire (un pointeur de plus) et il y a plus de liens à maintenir, donc plus de risques d'oublier une mise à jour.

5. Les listes circulaires

5.1 Circulaire simple

C'est une liste simplement chaînée où le dernier nœud ne pointe **pas sur NULL**, mais sur le **premier nœud**, ce qui forme un cercle.

```
head --> [3] --> [81] --> [19] --
          ^                      |
          ----------------------
```

Caractéristiques :

- Pas de début ni de fin « absolue ».
- Permet de parcourir la liste **en boucle**.
- Utile pour gérer des cycles (tour de jeu, file circulaire…).

Attention au parcours : il n'y a plus de `NULL` pour s'arrêter. Une boucle `while (temp != NULL)` tournerait **à l'infini**. On s'arrête quand on revient à la tête :

```
void afficherCirculaire(Node* head) {
    if (head == NULL) return;
    Node* temp = head;
    do {
        printf("%d -> ", temp->val);
        temp = temp->next;
    } while (temp != head);
}
```

On utilise un `do ... while` pour traiter le premier nœud avant de tester le retour à la tête.

5.2 Circulaire doublement chaînée

C'est une liste doublement chaînée où :

- le `next` du **dernier** nœud pointe vers le **premier**,
- le `prev` du **premier** nœud pointe vers le **dernier**.

On peut donc tourner dans les deux sens sans jamais rencontrer `NULL`.

6. Les erreurs classiques à éviter (très fréquentes en contrôle)

1. **Oublier `head = ...`** après un appel qui peut changer la tête (`head = insererDebut(head, x);`).
2. **Déréférencer `NULL`** : écrire `temp->next` alors que `temp` peut être `NULL`. Toujours tester avant.
3. **Mélanger les deux conditions de boucle** : `while (temp != NULL)` pour parcourir tout, `while (temp->next != NULL)` pour s'arrêter sur le dernier.
4. **Mauvais ordre des affectations** lors d'une insertion : d'abord `nouveau->next = temp->next`, ensuite `temp->next = nouveau`.
5. **Oublier `free`** lors d'une suppression (fuite mémoire), ou utiliser le nœud après l'avoir libéré.

6. **Perdre la tête** : avancer directement avec `head = head->next` pour parcourir. Utilise toujours un pointeur temporaire.
7. **Ne pas gérer les cas particuliers** : liste vide, un seul élément, suppression de la tête, valeur introuvable.
8. **Oublier de mettre `next = NULL`** dans un nouveau nœud (le dernier maillon doit toujours pointer sur NULL).
9. **Listes doubles** : oublier de mettre à jour un des quatre liens (`prev` ou `next` d'un des voisins).
10. **Listes circulaires** : utiliser `!= NULL` comme condition d'arrêt (boucle infinie).

7. Méthode pour réussir un exercice de liste chaînée

1. **Dessine** la liste avec des flèches avant d'écrire le code.
2. Identifie les **cas particuliers** : liste vide, un seul élément, début, fin.
3. Utilise un pointeur **temporaire** pour parcourir.
4. Pour insérer ou supprimer, repère le nœud **précédent**.
5. Fais les affectations de pointeurs **dans le bon ordre** (en dessinant à chaque étape).
6. N'oublie pas le **`return head`** et le **`free`** quand il le faut.
7. Relis ton code en le **traçant à la main** sur un petit exemple (2 ou 3 éléments).

8. Auto-test avant le contrôle

Essaie de répondre sans regarder, puis vérifie.

1. Quelle est la complexité de l'insertion au début d'une liste simple ? *(O(1))*
2. Pourquoi l'accès au i-ème élément est-il en O(n) ? *(il faut parcourir depuis la tête, les maillons ne sont pas contigus)*
3. Que vaut `head` pour une liste vide ? *(NULL)*
4. Pourquoi `insererDebut` retourne-t-elle un `Node*` ? *(la tête change, et la fonction travaille sur une copie du pointeur)*
5. Quelle est la condition de fin d'une liste simple ? (*`next == NULL`*)
6. Pour supprimer un nœud dans une liste simple, quel nœud faut-il trouver ? *(le précédent)*
7. Combien de liens modifier pour insérer un nœud au milieu d'une liste double ? *(quatre)*
8. Comment s'arrête-t-on dans une liste circulaire ? *(quand on revient à la tête, avec `do ... while (temp != head)`)*

9. Dans une liste doublement chaînée, que valent le `prev` du premier maillon et le `next` du dernier ? *(NULL et NULL)*
10. Quel est l'intérêt d'une liste doublement chaînée par rapport à une liste simple ? *(parcours dans les deux sens, et suppression sans devoir chercher le précédent)*

Exercice bonus : écris à la main les fonctions `compter`, `maximum` et `concatener` sans regarder, puis compare avec la section 3.12. Si tu y arrives, tu as compris le parcours d'une liste.

Bon courage pour lundi, tu es bien préparée.